

# The Bias-Variance Trade-off

---

## 1. What You'll Learn in This Section

---

In this lesson, you'll learn to:

- Explain why models must be judged on unseen test data instead of training data alone.
- Identify high bias (underfitting) and high variance (overfitting) from training and test error patterns.
- Use a validation dataset to pick the right polynomial degree, or feature count, for a model.
- Build and evaluate polynomial regression models in scikit-learn, sweeping degrees to find the trade-off zone.

## 2. Detailed Explanation

---

### Models, training, and testing

A machine learning model is just a learned function of the input, written  $\hat{f}(x)$ . You get it by feeding a training portion of your dataset to an algorithm — logistic regression, linear regression, SVM, a neural network, and so on. For simple linear regression, that function looks like  $\hat{f}(x) = w_0 + w_1 * x$ . Here  $w_0$  is the base value, also called the intercept: the value  $\hat{y}$  takes when  $x$  is zero. In mathematics this intercept is sometimes called a "bias

term" — a different idea from the bias-variance sense of bias covered below, even though both borrow the same word.

Once you have a model, you evaluate it on a separate test set using a metric suited to the task: F1 score or accuracy for classification, mean squared error (MSE) for regression. A model is expected to fit its training data reasonably well by default. The real goal is strong performance on data it has never seen. A model reporting 99% accuracy on training data alone tells you little, because it hasn't been checked against anything new. A model reporting 90% on held-out test data is far more trustworthy, because that data forces it to generalize. A model only earns its keep in production once it performs well on data it never trained on.

## The ice cream sales example

The running example throughout this lesson: an ice cream shop wants to predict daily sales from the day's outside temperature. Temperature is the input ( $x$ ) and sales is the output ( $y$ ), a continuous value — making this a regression problem (predicting an in-store visit scenario, with online delivery deliberately left out of the picture).

The assumed real pattern: sales are low at low temperatures and rise as temperature climbs toward a moderate range (roughly 25–35°C). They then plateau or dip slightly once it gets very hot (above roughly 35–40°C), because people prefer to stay indoors. Plotted, this shape rises, plateaus, and slightly falls — it looks like a quadratic (two-degree polynomial) curve, not a straight line.

## Increasing model complexity with polynomial features

A straight-line model,  $w_0 + w_1 * x$ , is degree one. When data does not follow a straight line, adding higher-degree terms of the same feature lets the model bend to fit curved patterns:

- Quadratic (degree 2):  $w_0 + w_1 * x + w_2 * x^2$
- Cubic (degree 3):  $w_0 + w_1 * x + w_2 * x^2 + w_3 * x^3$
- And so on for higher degrees.

These extra terms ( $x^2$ ,  $x^3$ ,  $x^4$ , ...) count as new features even though they're mathematically derived from the original one. They carry more information about the shape of the relationship without introducing a brand-new variable. The polynomial degree is simply the highest power present in the equation.

The same idea extends to classification. If two classes (drawn as crosses and circles) aren't linearly separable, a straight decision boundary from logistic regression misclassifies many points. A non-linear boundary, built from higher-degree terms like  $x_1^2$ ,  $x_2^2$ , and the interaction term  $x_1 * x_2$ , can wind around the data and classify it far more accurately. Neural networks are non-linear models by definition, which is part of why they can beat simpler linear models on complex, intermixed data. The point here is simply to show why non-linear boundaries exist, not to prove neural networks superior.

## Underfitting: high bias

If the polynomial degree is too low for the true pattern, the model can't capture that pattern even in the training data. Forcing a straight line onto data that is actually quadratic — like the ice cream example — leaves the fitted line far from most training points. The error (the vertical distance between each point and the line) is large across the board. This is high bias: the model is biased away from the true pattern before it's even tested on unseen data.

Watch out for: when training error is already high, you don't need to check test error separately. It's guaranteed to be no better, since the model never learned the underlying pattern in the first place.

## Overfitting: high variance

The opposite failure comes from adding too many polynomial terms. As degree climbs, the fitted curve starts bending to pass through nearly every training point — memorizing the data instead of capturing its shape. The result can look jagged, zig-zagging to chase individual points rather than the overall trend. Training error becomes very low, but when the same model meets test data, performance is poor: it overfit to the noise and quirks of the training set rather than the general pattern. This is high variance.

High variance isn't only a polynomial problem. Too many features in general — raw domain features like proximity to competing parlors or public-transport availability, not just polynomial-derived ones — can cause the same overfitting. This holds even though more information is usually desirable in principle.

## The trade-off

Neither extreme works:

| Failure mode                | Training error | Test error |
|-----------------------------|----------------|------------|
| High bias (too simple)      | High           | High       |
| High variance (too complex) | Very low       | Very high  |

The goal is a model that sits in between, where training and test performance are close to each other and both are acceptably low — not a model overfitting at one end or underfitting at the other.

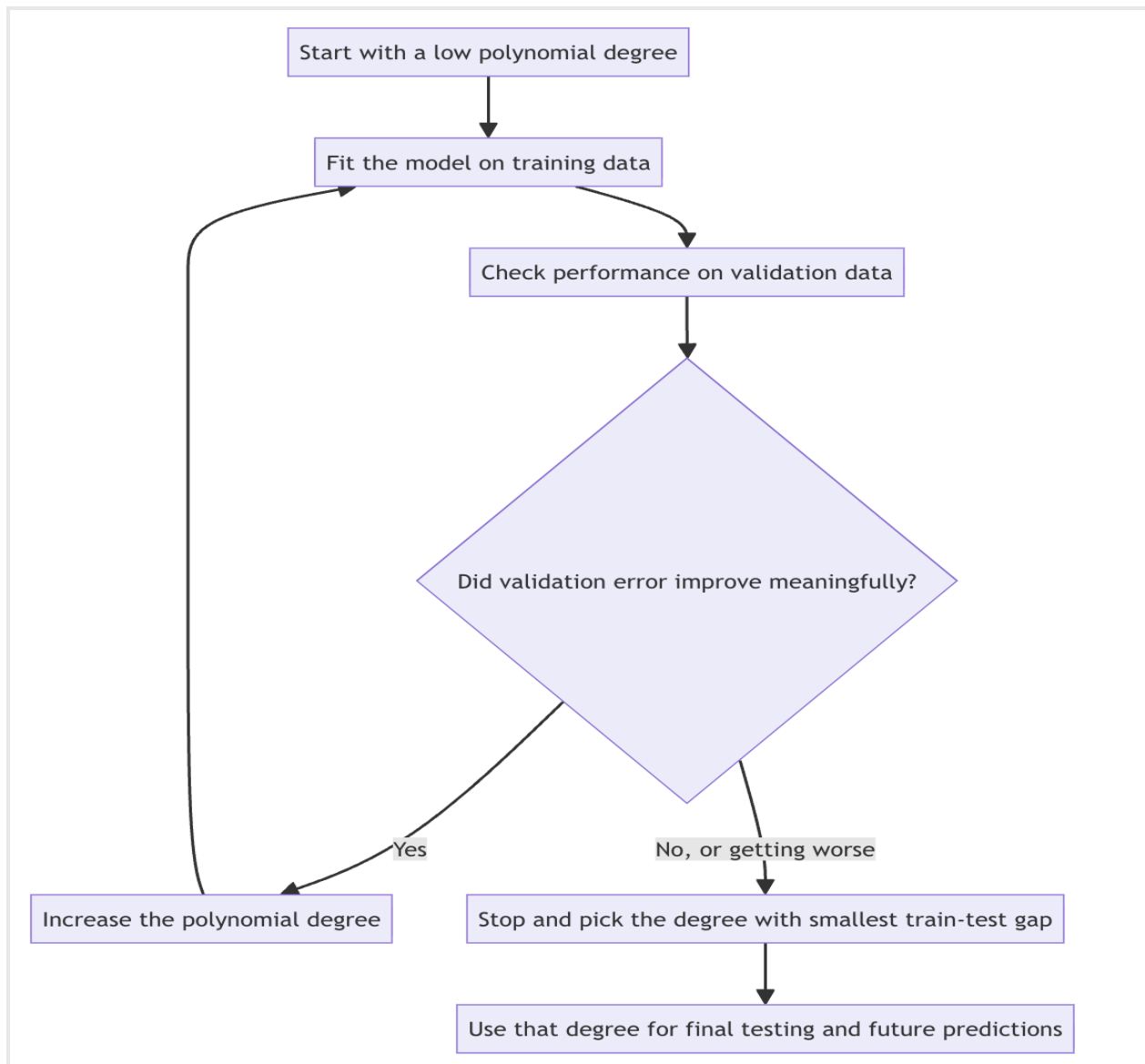
## Choosing the trade-off point: the validation dataset

Finding the right complexity (polynomial degree, or feature count) calls for a third split: the validation dataset. The full workflow uses three splits instead of two:

- Training data — the largest chunk, used to fit the model.
- Validation data — used to decide how complex the model should be.
- Test data — held out completely, used only for final, unbiased reporting, never for modeling decisions.

Typical ratios: 70/15/15, 60/20/20, or 80/10/10. These aren't fixed, but the training chunk is always largest, and validation and test are usually kept roughly equal to each other.

The procedure starts at a low degree (say, degree 1) and checks validation performance. If it's poor, the degree increases (say, to 2) and gets checked again. Complexity keeps increasing only as long as validation performance keeps meaningfully improving; once extra degree stops helping, or performance starts degrading, you stop and pick the degree with the best, most balanced validation result. Practically, this means computing the gap between training error and validation/test error across a range of degrees, then choosing the degree with the smallest gap, provided both errors are still reasonably low. This differs from just picking whichever degree gives the single lowest training error, which would simply select the most overfit model.



## Worked numeric demonstration

Using the ice cream temperature-vs-sales data, the model was fit at several degrees and scored by MSE on training and test data:

- Degree 1 (linear): training MSE  $\approx 1004$ , test MSE  $\approx 1097$ . Training error is already high — a clear case of high bias. The straight line can't capture the curved shape even on the data it was fitted to.

- Degree 2 (quadratic): both errors dropped substantially (test MSE around 67), since the quadratic curve matches the true shape of the relationship. This sits close to the desired balance point.
- Degree 15: training MSE dropped drastically, fitting almost every training point, but test MSE shot up to roughly 245 — several times worse than degree 2. This is high variance: excellent training fit, poor generalization.

A sweep across degrees 1 through 13 stored training MSE and test MSE for each degree. Training error steadily decreased as degree rose, while test error decreased only up to a point, then rose again. The single lowest test MSE across the sweep landed at degree 4 ( $\approx 65.4$ ) — only marginally better than degree 2, a difference too small to matter.

A gap column (test MSE minus training MSE) was computed for every degree. The gap was large at degree 1 ( $\approx 93$ ), dropped sharply at degree 2 ( $\approx 22.9$ ), stayed similarly low at degrees 3 ( $\approx 20.5$ ) and 4 ( $\approx 20.1$ ), then climbed again at higher degrees. Degrees 2 through 4 form the trade-off zone: training and test performance sit close together, and both are acceptably low.

## Code walkthrough in scikit-learn

The dataset was synthetically generated. Sixty temperature values were drawn uniformly between 10 and 40, and a true quadratic relationship ( $w_0 = 5$ ,  $w_1 = 4.2$ ,  $w_2 = -0.09$ ) generated sales. Random noise between 0 and 8 was then added, and sales were clipped at a lower bound of 0:

```
temperature = np.random.uniform(10, 40, 60)
true_sales = 5 + 4.2 * (temperature - 10) - 0.09 * (temperature - 10) ** 2
noise = np.random.uniform(0, 8, 60)
sales = true_sales + noise
sales = np.clip(sales, 0, None)
```

```
X = temperature.reshape(-1, 1)
y = sales
```

The small negative `w2` (-0.09) produces the slight plateau/drop at high temperatures; the large positive `w1` (4.2) drives the initial rise. `train_test_split` (from `sklearn.model_selection`) then splits the 60 points 70/30 into 42 training and 18 test instances.

`PolynomialFeatures` (from `sklearn.preprocessing`) generates polynomial terms up to a chosen degree. For one feature at degree 5, it produces `x`, `x^2`, `x^3`, `x^4`, `x^5`; for two features at degree 2, it produces `x1^2`, `x2^2`, and the interaction term `x1*x2`.

`make_pipeline` (from `sklearn.pipeline`) chains `PolynomialFeatures` with `LinearRegression` (from `sklearn.linear_model`) into one unit, so feature expansion and fitting happen together. `mean_squared_error` (from `sklearn.metrics`) computes MSE. A helper function bundles this workflow for a given degree:

```
def fit_and_evaluate(degree, X_train, y_train, X_test, y_test, plot=True):
    model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
    model.fit(X_train, y_train)

    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)
    train_mse = mean_squared_error(y_train, train_pred)
    test_mse = mean_squared_error(y_test, test_pred)

    # Generate 200 uniformly spaced points across the temperature range (10-40)
    # purely for smooth plotting of the model's predicted curve
    X_val_plot = np.linspace(10, 40, 200).reshape(-1, 1)
    val_pred = model.predict(X_val_plot)

    if plot:
        fig, ax = plt.subplots(figsize=(5.5, 4))
        ax.plot(X_val_plot, val_pred) # smooth predicted curve
```



```

        ax.scatter(X_train, y_train) # actual training points as
circles
        ax.set_xlabel("Temperature")
        ax.set_title(f"Degree {degree}, Train MSE {train_mse:.1f}, Test MSE
{test_mse:.1f}")

    return degree, train_mse, test_mse

```

These 200 extra points are purely for smooth plotting of the fitted curve — they are not a substitute for a real held-out validation set. Calling this function for degrees `[1, 2, 15]` reproduces the three worked examples above. A second loop sweeps degrees 1 through 13, collecting results into a DataFrame named `sweep`:

```

results = []
for d in range(1, 14):
    results.append(fit_and_evaluate(d, X_train, y_train, X_test, y_test,
plot=False))

sweep = pd.DataFrame(results, columns=["degree", "train_mse", "test_mse"])
best_row = sweep.loc[sweep["test_mse"].idxmin()]
sweep["gap"] = sweep["test_mse"] - sweep["train_mse"]

```

`idxmin()` returns the index of the smallest value in the `test_mse` column, and `.loc` retrieves that full row. The `gap` column is the practical way of locating the trade-off point — the degree with the smallest gap, where both errors stay acceptably low, is the one selected for delivery.

## Does more training data fix bias and variance?

Adding more training data is generally helpful, since it gives a model more information to learn from. But bias and variance are driven mainly by the number of features (and, for polynomial expansion, the degree) — not by the volume of data. The shape of the

pattern being learned doesn't change just because more points of the same pattern arrive; only new features change that shape. This resembles how a person's knowledge deepens as they absorb more information across chapters or weeks of study: more information deepens understanding, but it doesn't change the underlying subject being learned.

## Does raising the degree always trade bias for variance?

In general, yes. If a linear model shows high error on both training and validation data, that signals high bias. Adding more features — new domain features or derived polynomial terms — can move a model out of the high-bias zone and improve validation performance, up to a point. Beyond that point, more features risk pushing the model into high variance instead. Whether new features are genuinely independent domain signals or derived polynomial terms (like temperature-squared) is a modeling choice. Derived terms are technically dependent on the original feature but are treated as adding new, useful shape information.

## When a model fails after going live

A model that performs well during testing can still fail after deployment if it meets conditions outside what it was tested on. For example, a model trained on historical solar power and weather data from one site (say, Roorkee) may generalize well to unseen data from that same site or a very similar nearby location (like Saharanpur). It can still fail on a meaningfully different site (like Kurukshetra, Dehradun, or Mussoorie) whose conditions differ from training. This is a normal, expected part of a model's lifecycle. Client contracts for this kind of model typically run for years, during which the model gets monitored, retrained, and improved as new data, new sites, or new features appear, rather than delivered once and left untouched.

## Handling an already-large feature set

If a dataset already has seven features, blindly raising the polynomial degree on top of them raises the risk of landing in high variance before expansion even starts. The better approach is feature selection first: identify and keep only the best-performing subset of those seven features. One way to do this is using an ANOVA test to judge which features are more relevant to the regression or classification target. Only after that should higher-degree polynomial terms of the retained features be considered. There's no fixed rule for how many features or what degree is "too many" — it has to be determined experimentally, by checking training and validation performance at each step. This same experimental spirit applies to the broader question of whether a model can be completely free of both bias and variance. It's the ideal outcome, but not automatically guaranteed. How close a model gets depends heavily on how clean and representative the available data is — which is exactly why a validation dataset stays part of the workflow.

### 3. Key Takeaways

---

- A model is a learned function  $f(x)$ ; always judge it by performance on unseen test data, not training data alone.
- High bias means the model is too simple — training error itself is high, and test error is no better. High variance means the model is too complex — training error is very low, but test error is very high.
- Polynomial features ( $x^2$ ,  $x^3$ , ...) let a model capture curved, non-linear patterns, but pushing the degree too far causes overfitting.
- A validation dataset — separate from training and test data — is the tool for choosing model complexity. Track the gap between training and validation/test error across degrees, and pick the smallest gap where both errors stay acceptably low.

- Bias and variance are controlled by the number and type of features, not by the amount of training data. More data helps the model learn the existing pattern more reliably, but doesn't change the pattern itself.

Mental model: Think of a biased model as consistently missing the true target, while a high-variance model swings far and wide, chasing every fluctuation in the data rather than the general trend. The validation dataset is your compass — it guides you to the middle ground where training and test performance are close and both stay acceptably low.